

Báo cáo rà soát test code

<!-- framework: Jest | run_id: run_20260617_001004_391193 -->

Framework: Jest | Run ID: run_20260617_001004_391193

Tóm tắt kỹ thuật

- Framework: Jest
- Kết quả chạy: 12/12 test passed, coverage 83.33 %
- Vấn đề chính: Các test có assertion sai so với yêu cầu nghiệp vụ, dẫn đến việc xác nhận hành vi không đúng của hàm roundMoney và finalTotal.
- Artifact được xem xét: generated.test.js (test) và orderPricing.js (source).

Lỗi nghiêm trọng

- TC-003 – roundMoney:
 - Bằng chứng: expect(result).toBe(1); (test) vs yêu cầu “1.005 → 1.01”.
 - Ảnh hưởng: Kiểm thử xác nhận giá trị sai, che khuất lỗi thực tế của hàm roundMoney (đang làm tròn xuống do lỗi floating-point).
 - Cách sửa: Thay đổi kỳ vọng thành toBeCloseTo(1.01, 2) và sử dụng hàm roundMoney chính xác (có thể cải tiến bằng Number((value).toFixed(2))).
- TC-004 – roundMoney:
 - Bằng chứng: expect(result).toBe(1.0); nhưng yêu cầu “1.004 → 1.00” (đúng) – tuy nhiên test vẫn dùng toBe(1.0) thay vì toBeCloseTo(1.00, 2).
 - Ảnh hưởng: Kiểm thử phụ thuộc vào so sánh số nguyên, có thể gây false positive nếu hàm thay đổi cách làm tròn.
 - Cách sửa: Dùng toBeCloseTo(1.00, 2).
- TC-011 – finalTotal (VIP order):
 - Bằng chứng: expect(total).toBe(337); trong khi tính toán đúng theo yêu cầu là 347.
 - Ảnh hưởng: Kiểm thử xác nhận giá trị sai, làm cho lỗi trong finalTotal không được phát hiện.
 - Cách sửa: Cập nhật kỳ vọng thành toBe(347) (hoặc toBeCloseTo(347, 2)).
- TC-012 – finalTotal never negative:
 - Bằng chứng: expect(total).toBeCloseTo(31.4, 2); nhưng kết quả thực tế theo logic là 26.4.
 - Ảnh hưởng: Kiểm thử không phản ánh đúng quy tắc “never negative”, gây hiểu lầm về hành vi hàm.
 - Cách sửa: Thay đổi kỳ vọng thành toBeCloseTo(26.4, 2) và kiểm tra giá trị không âm (toBeGreaterThanOrEqual(0)).
- Import/Export không nhất quán:
 - Bằng chứng: Test thêm fallback cho roundMoney vì module không export nó.
 - Ảnh hưởng: Khi source thay đổi (ví dụ export roundMoney), fallback có thể gây xung đột hoặc che khuất lỗi.
 - Cách sửa: Cập nhật orderPricing.js để export roundMoney (và shippingFee nếu cần) và loại bỏ fallback trong test.

Test còn thiếu

- Kiểm thử cho shippingFee (đúng/ sai tham số, các mức phí, lỗi destination không hỗ trợ).
- Kiểm thử biên cho calculateDiscount khi subtotal = 0 (đảm bảo không ném lỗi).
- Kiểm thử cho finalTotal với storeCredit lớn hơn tổng (đảm bảo trả 0).
- Kiểm thử cho finalTotal khi shippingFee trả giá trị lớn (đảm bảo không gây overflow).

Assertion yếu

- TC-003 & TC-004: Sử dụng toBe với số thực, không chịu lỗi floating-point.
- TC-011 & TC-012: Kỳ vọng giá trị cố định sai, không có kiểm tra toBeGreaterThanOrEqual(0) cho trường hợp không âm.
- TC-010: Kiểm tra lỗi đúng, nhưng không kiểm tra thông điệp lỗi chi tiết (đã làm nhưng tốt hơn nên dùng regex).

Rủi ro maintainability

- Mocking shippingFee ở mức module (jest.spyOn(target, 'shippingFee')) có thể gây side-effect nếu các test khác phụ thuộc vào thực thi thực tế của shippingFee.
- Fallback implementation cho roundMoney trong test làm tăng độ phức tạp và có thể bị lãng quên khi source cập nhật.
- Hard-coded giá trị kỳ vọng (337, 31.4) không dựa trên công thức, dễ lỗi khi logic thay đổi.

Gợi ý sửa ngay

1. Sửa các kỳ vọng sai trong TC-003, TC-004, TC-011, TC-012 theo công thức và dùng toBeCloseTo.
2. Export roundMoney (và shippingFee nếu muốn test trực tiếp) từ orderPricing.js, sau đó xóa fallback trong test.
3. Thêm test cho shippingFee bao gồm các trường hợp: trọng lượng ≤ 0 , destination không hợp lệ, phí tăng theo trọng lượng và fragile.
4. Cập nhật TC-012 để kiểm tra expect(total).toBeGreaterThanOrEqual(0).
5. Sử dụng matcher toThrowErrorMatchingSnapshot hoặc regex để kiểm tra thông điệp lỗi một cách linh hoạt.
6. Rà soát lại toàn bộ test để đảm bảo mọi kỳ vọng đều dựa trên công thức tính toán đã mô tả trong yêu cầu.